USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

The Note Pad, and a Workaround Wearing a Costume

UM-NATOS-022

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-022	1.1 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

The first thing in this project that takes **text** from a person and keeps it. Everything before it read a tap or a serial line, and a serial line needs a computer attached — which is the thing the launcher exists to avoid needing.

A message is written on a multi-tap keypad, saved to flash, and read back after a power cycle. That last part is the whole claim, and it is verified in §6.

The report's more useful half is about the keypad. It looks like a design choice — a phone keypad, on a device that resembles a phone — and it is not. It is a **workaround for a touch calibration fault**, adopted because the obvious layout was unusable.

> **Revision 1.1 — the fault has since been corrected.** UM-NATOS-017 §7.4 replaced the corner-derived constants with an on-device calibration, and §3.2 below is retained as written because *the diagnosis in it was wrong* and the way it was wrong is the useful part. The keypad remains, now as a choice rather than a necessity.

2. Why this is native code

Every other application here is VM bytecode confined to a 26-row strip (UM-NATOS-016 §2). This is native, in the kernel, owning the launcher's region like the 3D view does.

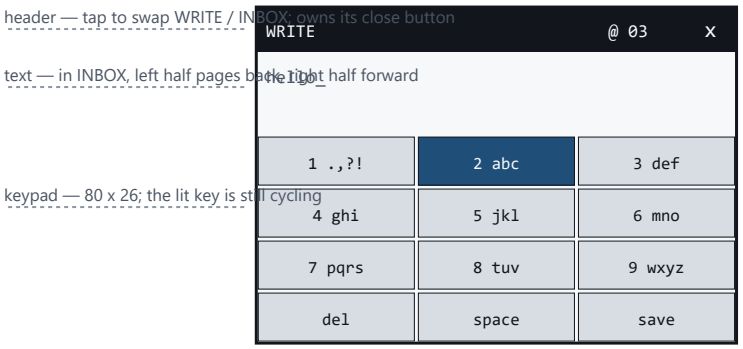
Two reasons, and neither is a virtue:

- a keyboard does not fit in 26 rows
- the VM has no syscall that returns a keypress

So **text entry is a kernel feature rather than a service applications can request**. Any program wanting text input cannot have it. The version that would fix this is a keyboard syscall alongside `SYS TOUCH`, and it is not built.

That is recorded here and in `notes.h` because it is the kind of limitation that becomes invisible once the feature works: the note pad takes text, so text entry appears solved, and it is solved for exactly one program.

3. The keypad is a workaround



Keys are 80 px wide because the touch mapping reads about 24 px low on X. That fault is not fixed — it is absorbed. A 24 px key was destroyed by it: typing `e` produced `w`, reliably.

Figure 1. *The note pad. Key size is a consequence of a touch calibration fault the layout absorbs rather than corrects.*

3.1 The layout that failed

The first version was QWERTY: three letter rows, keys **24 × 26**.

It was close to unusable. Tapping `e` produced `w` — one key to the left, consistently, for every key. Reported as "*quite a task to write anything legible*", which is an accurate description of a keyboard whose letters are reliably wrong.

3.2 What is actually broken

The touch mapping reads systematically about **24 px low on X**.

The calibration constants in `touch.c` were derived by tapping the four corners of the glass (UM-NATOS-017 §7.1). A finger cannot reach the extreme corner of a bezelled panel, so the observed range is narrower than the true one, and everything maps inward — the error growing with distance from centre.

Fixing it properly means calibrating from targets **in the middle of the screen**, where the error is measured rather than extrapolated from the one place a fingertip cannot go.

> **What that paragraph got wrong.** The cause is right; the characterisation is not. "Reads systematically about 24 px low on X" describes a constant offset, and the fault was a **magnification** — the narrow range is a divisor, so the error is proportional to distance from centre and changes sign across the screen (−29 px at the left edge, +12 px at the right; UM-NATOS-017 §7.4).
>
> The number 24 was real. It was measured in one place, on keys near the middle, and then written down as though it were a property of the panel rather than of where it happened to be measured. A single sample of a position-dependent quantity looks exactly like a constant.
>
> It also explains something this document treated as unremarkable: no adjustment of the constants ever helped, because there is no offset that corrects a scale error. That should have been the clue.

3.3 Why the keypad works anyway

Twelve keys, 3 × 4, each **80 × 26** — a third of the panel wide. An 80 px key absorbs a 24 px error. A 24 px key is destroyed by it.

The launcher's icon cells are also 80 px and were never affected, which is why this fault survived unnoticed until something needed fine positioning.

> The keypad is tolerant of the defect, not a correction of it. Those are different, and calling the first one a fix is how a known fault becomes folklore. Anything finer than 80 px will hit it again.

That paragraph is the reason this section survived. Because the keypad was never called a fix, the fault stayed on the record as unresolved, and calibrating it properly stayed on the list instead of quietly becoming the way things are. A workaround that is honestly labelled is a debt; one that is called a solution is a defect with good manners.

Since correction, 80 px keys are no longer load-bearing. They are kept because multi-tap is the point of this app, not because a smaller key cannot be hit — and that difference is now testable rather than assumed.

The cost is real and worth stating: three taps for `c` against one tap on a key that cannot be hit. While the fault stood, slow and correct beat fast and wrong, and that trade was forced by a bug rather than chosen. It is now chosen — the same keypad, held for a different reason, which is worth writing down because a decision whose original justification has expired is one nobody re-examines.

3.4 Multi-tap details that matter in use

- **The live key is drawn back-lit.** Without it, multi-tap is guesswork about whether a press registered — and the press that "did not register" is usually one that did, replacing the letter you wanted.
- **An 800 ms timeout commits.** This is how two letters from one key are typed.
- **Cycling wraps** rather than sticking at the end of a sequence.
- **One press, one action**, latched on the first sample (UM-NATOS-021 §4.2). The last sample before release is the one a resistive panel gets wrong, and typing the *wrong* letter is worse than missing a press.

4. Storage

4.1 A separate sector, deliberately

Messages live in the flash sector **after** the kernel's boot record, not in it.

A flash write is erase-then-write over a whole 4 KB sector. Sharing one would mean rewriting the boot counter on every save, and losing every message if a save were interrupted while the counter was the thing being rewritten.

Separate sectors can only damage themselves.

4.2 Eight messages of 160 characters

160 is the SMS limit. It suits what this is imitating, and it bounds the store: eight of them plus a header is under 1.4 KB, comfortably inside one sector.

When full, the oldest is dropped. A note pad that refuses to save is worse than one that forgets what you wrote first.

Validation follows the boot record's rule: magic, version, count and checksum, and on any failure the store resets to empty — so a first run and a corrupt sector behave identically rather than producing a half-believed store.

5. Reading

The **header bar is the navigation control**. Tapping it swaps `WRITE` and `INBOX`, the way a phone with three buttons did it, and it costs no key.

In the inbox, the left half of the text area pages back and the right half forward, with a position indicator so paging has a place rather than being an endless cycle.

Letter keys do nothing while reading. A stray tap must not silently begin composing over a message being read. `del`, `space` and `save` still act, so there is always a way out of a state.

6. Verification

```
write a message on the panel
save                        -> header shows "saved", box clears
power cycle
reflash
boot                        -> messages : loaded 1 saved
open notes, tap header      -> message reads back in the inbox
```

The reflash matters: it proves the message is in flash rather than in RAM that happened to survive a warm reset.

The box clears **only on success**. A save that failed must not look like one that worked by leaving an empty box behind.

7. The close button nobody drew

Worth its own section because it is a fix's blast radius landing on a view that did not exist when the fix was written.

UM-NATOS-021 §6.5 stopped the 3D view's close button flickering by stamping it into the raycaster's framebuffer, and made `desktop_chrome()` skip drawing it whenever the framebuffer is on. Correct for the only full-region view that then existed.

The note pad has no framebuffer to stamp into. It fell through both paths: **present, hit-testable, and drawn by nobody.** The button worked — tapping the corner returned to the launcher — and was invisible.

Ownership is now explicit for all three cases:

view	who draws the close button
3D, framebuffer on	the raycaster stamps it into the buffer
3D, framebuffer off	`desktop_chrome()`, and it flickers
note pad	the app, in its own header

The note pad draws it at exactly the coordinates `desktop_chrome_touch()` already tests, so the drawing and the hit test sit in one file and cannot drift. That was an open gap in UM-NATOS-021 §9 and is now half closed — half, because nothing *enforces* the agreement, it is merely adjacent.

8. Metrics

Quantity	Value
Keys	12, 80 × 26
Key size that failed	24 × 26
Touch X error absorbed	~24 px near centre, up to 29 px at an edge
Calibration status	corrected, UM-NATOS-017 §7.4
Multi-tap commit timeout	800 ms
Messages stored	8 × 160 characters
Store size	~1.4 KB in one 4 KB sector
Compose buffer	256 B in `.bss`
Flash sectors used	1, separate from the boot record

9. What this does not establish

- **The touch calibration is corrected but still linear.** §3.2 and UM-NATOS-017 §7.4. It has been run on one unit once, and nothing measures panel non-linearity between the target positions, so a UI element much finer than the keys has not been shown to work — only stopped being impossible.
- **The full store has never been exercised.** Eight messages have not been saved, so the oldest-dropped path has never run outside reasoning.
- **Corrupt-sector rejection is reasoned, not tested.** The checksum path has never seen a torn write, exactly as UM-NATOS-018 §8 says of the boot record.
- **A save interrupted by power loss is untested.** The design says the store reverts to the previous contents; nothing has pulled power mid-erase to confirm it.
- **No editing.** `del` removes the last character. There is no cursor movement, no insertion, and no way to change a saved message.
- **No timestamps.** Messages have no date, because the kernel has no clock that survives a power cycle — only a tick counter that restarts at zero.
- **Text entry is not available to applications.** §2.

10. References

- UM-NATOS-017 §7.1 — the corner calibration this inherits its error from, and §7.4 — the on-device calibration that corrected it
- UM-NATOS-018 — the flash record, and the sector this store sits beside
- UM-NATOS-021 §4.2 — first-sample latching, and §6.5 — the flicker fix that caused §7
- UM-NATOS-016 §2 — the viewport model that makes a keyboard impossible for a VM program
- `kernel/notes.c` — keypad, multi-tap, the two views
- `kernel/messages.c` — the flash-backed store